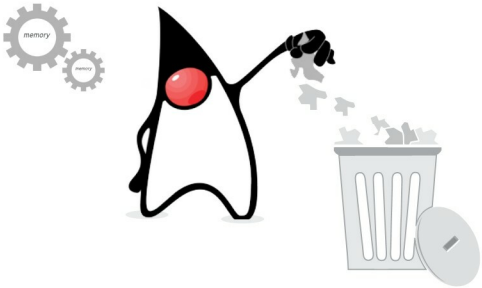
A Sneak Peek into the World of Garbage Collection

This is not about hygiene and sanitation. In Java, garbage refers to unreferenced objects. Java memory can be made more efficient by removing these objects. Garbage collection is the process of reclaiming the runtime unused memory, automatically.



There was a time when developers were under constant pressure to write efficient code that kept a check on the memory usage and cleaning up of unused or idle resources. This constraint not only added to the woes of the developer but also made the code lengthy and repetitive. Java, the object orient language, came up with a brilliant mechanism for garbage collection. It eased the load on coders by running a daemon thread that automatically cleaned the memory as and when required and made it available for the program.

Now, the question is: where does Java keep the object? Well, the answer is that since Java abides by the OOPs paradigm, it stores everything as an object in the Heap memory. Whenever objects are created, they occupy some space in the Heap and are further referenced as per usage. From time to time, JVM marks objects that are idle for a long time or are not referenced, and selects them for deletion. Garbage collection in Java is tracking down all the objects that are still used, and marking the rest as garbage.

Before getting into garbage collection, let's understand the memory pool of Java, i.e., how the Heap is divided into different segments and where objects are present within the Heap at different times. To start with, the Heap is divided into the Young and Tenured generations, along with the PermGen space. The Young generation is further divided into Eden, Survivor1 and Survivor2. When created, objects reside in Eden; if garbage collection (GC) does not result in sufficient memory space inside Eden, then the object is allocated to the Old generation. When garbage is being collected from Eden, the GC runs from the root to all reachable objects and marks them as alive—this process is called marking. After marking, all the live objects are copied from Eden to Survivor spaces, after which the whole of Eden is empty and can be reused to allocate more objects. This entire process is named as Mark and Copy. Beside Eden lie the Survivor spaces called 1 and 2, one of which is always empty. It is this empty space that will start getting filled once the GC works on Eden.

The copying of live objects in Survivor space is repeated several times, until some objects are considered mature enough and are promoted to the Old generation, where they reside until they become unreachable. In contrast to the Young generation, the size of the Old generation is quite large, and the frequency of GC here is less than in the Young generation. When GC takes place in the Old generation, the following takes place: a) Reachable objects are marked by setting the marked bit next to all objects accessible through GC roots; b) All unreachable objects are deleted; and c) The content of old space is compacted by copying live objects contiguously